

L2: Solving Problems by Searching

- Tree vs Graph search (graph keeps track of visited nodes, more space cost)

Uniformed Search Algorithms (no clue on how close a state is to the goal)

BFS	Explores level by level, uses early goal test. Equivalent to UCS if costs are uniform.	Queue
UCS	Expands least-cost unexpanded node, always finds the shortest path.	Priority Queue
DFS	Similar to BFS, but explores deeper paths first.	Stack
DLS	DFS with a depth limit, backtracks if limit is reached.	Stack
IDS	Repeatedly applies DLS, increasing the depth limit until a solution is found.	Stack with depth

Informed Search Algorithms (has a clue on how close a state is to the goal)

- Heuristic**, $h(n)$: Estimated cost from n to goal
 - Admissible** (relaxed): $h(n) \leq h^*(n)$, A^* tree search is optimal (assume $h(G) = 0$) if admissible
 - Consistent**: $h(n) \leq c(n, a, n') + h(n')$ and $h(G) = 0$, consistent $\rightarrow$ admissible, $\sim$ admissible $\rightarrow$ consistent (contrapositive), A^* graph search is optimal if consistent
 - Dominance**: $h_2(n) \geq h_1(n)$ (admissibility is not a requirement for dominance, but if inadmissible not useful)
- | | | |
|------|--|----------------|
| GBFS | Expands node with lowest $h(n)$, BFS same as best-first search with $f(n) = \text{depth}$ | Priority Queue |
| A* | Expands node with lowest $f(n) = g(n) + h(n)$, $g(n)$: path cost to reach node n , optimal if h admissible despite negative edge cost, won't work with negative cycle, not optimal/complete on graphs with negative heuristics | Priority Queue |

	BFS	UCS	DFS	DLS	IDS	GBFS	A*
Complete	Yes, if tree is finite	Yes, if $\epsilon > 0$ and C' is finite	No, when depth is infinite (loops)	No	Yes, if b is finite	No	Yes
Time	$O(b^d)$	$O(b^{C'/\epsilon})$	$O(b^m)$	(b^l)	$O(b^d)$	$O(b^m)$, good heuristic gives improvement	$O(b^m)$, good heuristic gives improvement
Space	$O(b^d)$, worst case last expand last child in a branch	$O(b^{C'/\epsilon})$, C' optimal cost, ϵ minimum edge cost	$O(bm)$, m maximum depth	$O(bl)$, l depth limit	$O(bd)$, if used with DFS	$O(b^m)$	$O(b^m)$
Optimal	Yes, if step cost is the same everywhere	Yes, if $\epsilon > 0$ ($\epsilon = 0$ may cause zero cost cycle)	No	No, if used with DFS	Yes, if step cost is the same everywhere	No	Depends on the heuristics

L3: Local & Adversarial Search

Local Search Algorithms

- Goal: find good enough solutions efficiently, focus on solution state, not search path

- HILL-CLIMBING (GREEDY LOCAL SEARCH)**

Moves to neighbour with highest value. Challenges: local maxima, ridges (sequence of local maxima), plateaus (sequence of same value local maxima), shoulders (same value, but progress is possible). Escape techniques are:

- Tabu Search**: Avoid revisiting tabu states, but overtime algorithm removes older states from tabu list, allowing them to be revisited if necessary
- Random Restarts**: Restart with a new initial state, repeated multiple times and select best solution
- Random Walk**: Occasionally make random moves based on probability

- SIMULATED ANNEALING**

Allows "bad moves" based on probability $P(\text{current}, \text{next}, T) = e^{\frac{\text{value}(\text{next}) - \text{value}(\text{current})}{T}}$; reduce T gradually helps guarantee finding global maxima with high probability

Adversarial Search Algorithms

Solve games with two players (fully observable, discrete, terminal state exists, two-sum, turn-based)

Minimax	Optimal opponent assumed. MAX maximizes score, MIN minimizes MAX's score. Use cutoff for max depth, returns $\text{eval}(\text{state})$ when terminal state or max depth is reached
Negamax	Simplifies minimax by maximizing the negative evaluation score $\text{negamax}(\text{state}) = \max(-\text{negamax}(\text{opp's state}))$
Alpha-beta	Prune branches to improve efficiency, $O(b^{\frac{3}{2}})$ with perfect ordering α = highest value for MAX, β = lowest value for MIN - Stop at MIN node if $\leq \alpha$, or at MAX node if $\geq \beta$ - Update $\alpha = -\alpha$, $\beta = \beta$ recursively at nodes (α at MAX nodes, β at MIN nodes)

L4: Intro to ML & Decision Trees

Types of Problems

- Regression**: predict continuous output (e.g. temperature: 0.67)
- Classification**: predict discrete output (e.g. animal: cat vs dog)

Types of Feedback

- Supervised**: Agent learns from being given the right answers for each input **Formalism**
 - In supervised learning, we assume that y is generated by a **true mapping function** $f: x \rightarrow y$.
 - We want to find a **hypothesis** $h: x \rightarrow \hat{y}$ (from a **hypothesis class** H) such that $h \approx f$ given a **training set** $\{(x_1, f(x_1)), \dots, (x_n, f(x_n))\}$.
 - We use a **learning algorithm** to find this hypothesis
- Unsupervised**: No feedback/answer is given to the agent
- Weakly supervised**: Answer given to the agent but they are not precise
- Reinforcement**: Agent learns from a series of reinforcements, rewards and punishments ("trial and error")

Performance Measure

- Regression: Mean Squared/Absolute Error**

$MSE = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2$, $MAE = \frac{1}{n} \sum_{i=1}^n |\hat{y}_i - y_i|$
Note: **MSE is more volatile to outliers** as it penalises error more by squaring it

- Classification: Correctness & Accuracy**

Correctness: $\hat{y} = y$, Accuracy = $\frac{1}{N} \sum_{i=1}^N \mathbb{1}_{\hat{y}_i = y_i}$ (1 if correct, else 0)

- Classification: Confusion Matrix**

- TP**: actual positive predicted positive, **FP**: actual negative predicted positive,
- TN**: actual negative predicted negative, **FN**: actual positive predicted negative

K-Means Clustering

- Algorithm** (note: centroid is the mean of the data points in a cluster):
 - Randomly initialize K centroids (choose K using elbow method (loss vs K) / application dependent)
 - Assign each sample to cluster by nearest centroid (have deterministic tiebreaker)
 - Recompute centroids with new clusters (repeat 2-3 until convergence)

$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN}, \text{Precision} = \frac{TP}{TP + FP}, \text{Recall} = \frac{TP}{TP + FN}, F1 \text{ Score} = \frac{2}{\frac{1}{\text{Precision}} + \frac{1}{\text{Recall}}}$$

Precision: How many selected items are relevant? (Maximise if FP is very costly, e.g. email spam, satellite launch data prediction)

Recall: How many relevant items are selected? (Maximise if FN is very dangerous, e.g. cancer prediction but not music recommendation)

Decision Tree Learning

Entropy: A measure of randomness in set of data

$$I(P(v_1), \dots, P(v_k)) = -\sum_{i=1}^k P(v_i) \log_2 P(v_i)$$

For a data set containing p positive and n negative example:

$$I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) = -\frac{p}{p+n} \log_2\left(\frac{p}{p+n}\right) - \frac{n}{p+n} \log_2\left(\frac{n}{p+n}\right)$$

Information Gain: Reduction in entropy from attribute test

- An attribute A divides the training set E into subsets $E_1, \dots, E_r$ according to their values for A , where A has r distinct values
- Hence, the **information gain** = entropy of this node $\sim$ total entropy of children nodes

$$\text{remainder}(A) = \sum_{i=1}^r \frac{p_i + n_i}{p + n} I\left(\frac{p_i}{p_i + n_i}, \frac{n_i}{p_i + n_i}\right), IG(A) = I\left(\frac{p}{p+n}, \frac{n}{p+n}\right) - \text{remainder}(A)$$

Other issues:

- Continuous-valued attributes**: partition the values into a discrete set of intervals
- Missing values**: assign most common value of attribute / assign most common value of attribute with the same output / assign the probability to each possible value and sample / drop the attribute / drop the rows / etc

Handling Inconsistent Data:

- Pruning** to remove unnecessary branches and nodes, creating simpler decision tree
 - Pre-processing** of data to remove outliers that create noise
 - Select only relevant features**, so less relevant features that could create inconsistencies are not part of the decision tree
 - Collect more data on new features** to clearly differentiate the inconsistent classes
- Overfitting**
- Decision tree pruning** results in a **smaller tree** which may have **higher accuracy**
 - Min-sample**: when sample number doesn't hit the minimum, merge until it is greater than or equal to the minimum
 - Max-depth**: when it is over the maximum depth, merge into one node
 - Further simplification**: if one of the children is fully T/F and the other child is majority T/F respectively, then can merge

L5: Linear Regression

$$J_{MSE}(w) = \frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2, \frac{\partial J_{MSE}(w)}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

$$J_{MSE}(w) = \frac{1}{2m} \sum_{i=1}^m (w^T x^{(i)} - y^{(i)})^2, \frac{\partial J_{MSE}(w)}{\partial w_j} = -\frac{1}{m} \sum_{i=1}^m (w^T x^{(i)} - y^{(i)}) x_j^{(i)}, \text{Gradient: } \begin{bmatrix} \frac{\partial J(w)}{\partial w_0} \\ \frac{\partial J(w)}{\partial w_1} \\ \frac{\partial J(w)}{\partial w_2} \\ \vdots \end{bmatrix}$$

Note: Gradient always points into the direction of the greatest increase of the function

Gradient Descent

- Used to find weights to minimize loss function (where gradient = 0)
 - Start with some w
 - Pick a nearby w that reduces $J(w)$: $w_j \leftarrow w_j - \gamma \frac{\partial J(w)}{\partial w_j}$
 - (calculate partial derivatives first, before updating new weights)
 - Repeat until minimum is reached (doesn't guarantee global minimum)
- If γ is too large, loss exponentially increases due to the huge random movements. If γ is too small, takes too long for loss to converge. For large number of epochs/iterations, have decreasing learning rate, smaller gradient, smaller step
- Note: MSE loss function is convex for linear regression (only one minimum)

Batch	Use all training examples
Mini-batch	Use a subset of training examples + Cheaper / iteration + Randomness may escape local minima
Stochastic	Select one random data point at a time + Cheapest / iteration + More randomness escapes local minima

Linear Regression: Challenges and Solutions

- Too many attributes: $h_w(x) = \sum_{j=0}^n w_j x_j = w^T x$ (dot product)
 - Features of Different Scales: Mean normalization: $x_j \leftarrow \frac{x_j - \mu_j}{\sigma_j}$
(Others: Min-max scaling, Robust scaling, Different learning rate for each weight)
 - Non-Linear Relationship: Polynomial regression using transformed features (need to scale too)
- Normal Equation**
- Assuming $X^T X$ is invertible, $w = (X^T X)^{-1} X^T Y$
 - Note: Using normal equation is slow for large number of features: $O(n^3)$
 - $X^T X$ is non-invertible when it is singular which occurs when its rows or columns are linearly dependent (collinear/duplicate data or highly correlated features)
 - Solutions: using gradient descent, apply regularization techniques (Ridge/Lasso regression), Pseudoinverse (Moore-Penrose pseudoinverse)

L6: Logistic Regression

- Use logistic (sigmoid) function to give continuous value output used for classification
- MSE loss function is non-convex for non-linear regression
- MSE loss function is non-convex for non-linear regression

L6: Logistic Regression

- Use logistic (sigmoid) function to give continuous value output used for classification
- MSE loss function is non-convex for non-linear regression
- Use binary cross-entropy as loss function, BCE is convex for logistic regression

$$\sigma(z) = \frac{1}{1 + e^{-z}}, h_w(x) = \sigma(w_0 + w_1 x_1 + \dots + w_n x_n), w_j \leftarrow w_j - \gamma \frac{\partial J(w)}{\partial w_j}$$

$$J_{BCE}(w) = -\frac{1}{n} \sum_{i=1}^n y^{(i)} \log(h_w(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_w(x^{(i)}))$$

$$\frac{\partial J_{BCE}(w)}{\partial w_j} = \frac{1}{n} \sum_{i=1}^n (h_w(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

Logistic Regression: Challenges and Solutions

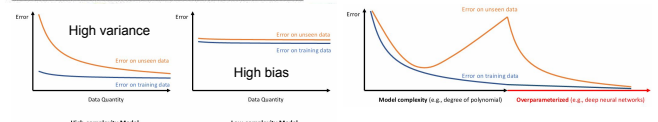
- Too Many Attributes: $h_w(x) = \sigma(w^T x)$
- Local optima problem: run multiple times and choose clusters with minimum loss or use smarter initialization (e.g., **K-Means++**: choose next centroid farthest from existing ones or with weighted probability based on squared distances)

Non-Linear Decision Boundary: transform features

Multi-class Classification

- One-vs-All**: Focus on one class each time (A or not A, B or not B, etc), then pick highest probability for that input among the different hypotheses
- One-vs-One**: Choose a pair of classes each time (A and B, A and C, etc), then pick class with most wins (frequency of highest probability in each pair)

Model Evaluation & Selection: Bias and Variance



- In general, increasing data points used for training does not improve bias, but decreases variance (also lower degree higher bias, higher degree higher variance) use regularization to reduce overfitting and reduce testing error

(Low) Bias => if more data points help see ground truth

(Low) Variance => retraining with diff data set leads to same model

Models with high bias & high variance:

- Weak dummy classifiers not trained to optimize a loss func (e.g. max val)
- Model trained with significant amt of outliers from a random distribution distinct from training data

Hyperparameter tuning: Grid/Exhaustive search, Random search, Local search, Others (successive halving, Bayesian optimization)

L7: Support Vector Machines

Regularization

- Ridge Regression heavily penalizes larger parameters, to pull all parameters towards smaller values while Lasso Regression zeros out certain parameters, implicitly selecting features to be 'excluded' and behaves as a feature selection strategy

Linear Regression with Regularization:

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n |w_j|^2 \text{ (L1-Norm: Lasso Regression)}$$

$$J(w) = \frac{1}{2m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)})^2 + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2 \text{ (L2-Norm: Ridge Regression)}$$

$$w_j \leftarrow w_j - \gamma \frac{1}{m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)}) x_j^{(i)} - \gamma \frac{\lambda}{m} w_j$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

$$w = (X^T X + \lambda I)^{-1} X^T Y$$

Logistic Regression with Regularization

$$J(w) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log(h_w(x^{(i)})) + (1 - y^{(i)}) \log(1 - h_w(x^{(i)})) + \frac{\lambda}{2m} \sum_{j=1}^n w_j^2$$

$$w_j \leftarrow w_j - \gamma \frac{1}{m} \sum_{i=1}^m (h_w(x^{(i)}) - y^{(i)}) x_j^{(i)} - \gamma \frac{\lambda}{m} w_j$$

SVM with Kernel Trick

- When data is not linearly separable, map features to higher dimension, then use Kernel to replace the dot products between two transformed features

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T Y$$

$$w = (X^T X)^{-1} X^T$$

L8 & 9: Intro to Neural Networks

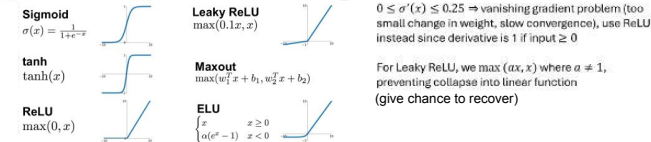
Perceptron Learning Algorithm

1. Initialize all weights w
2. Select a misclassified sample and update w . Repeat until convergence/max steps reached: $w \leftarrow w + \gamma(y^{(i)} - \hat{y}^{(i)})x^{(i)}$

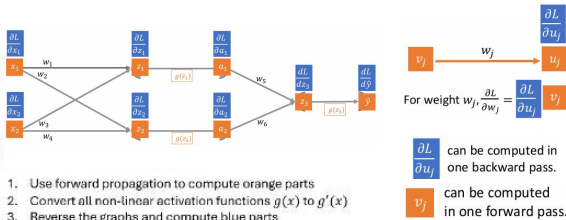
- If data is not linearly separable, algorithm will not converge
- Reordering data points can help converge faster, but may have different weight
- Perceptron only uses sign function as the activation function

(Single Layer / Multi-Layer) Neural Network

- Activation function (usually non-linear):



- **Single-layer NN** can only be used for linearly separable model, uses gradient descent to determine weights (note: derivative of sigmoid $\sigma'(x) = \sigma(x)(1 - \sigma(x))$), improve performance by doing feature mapping (might lead to overfitting)
- **Multi-layer NN** used for more complex model (non-linearly separable), can improve performance by increasing depth or width of hidden layer (might lead to overfitting)
- **Regression:** linear/no activation (for multi-layer collapses into simple linear model)
- **Binary Classification:** sigmoid activation
- **Multi-class Classification** (C classes): softmax activation $g(x) = \frac{e^{x_i}}{\sum_{j=1}^C e^{x_j}}$
- $g(x_i)$ = probability of being classified as class $i \Rightarrow \sum_{i=1}^C \hat{y}_i = 1$ $g(x_1) + g(x_2) \dots + g(x_C) = 1$
- **NN vs Other Methods (Logistic Regression/SVM):** has learned feature mapping (using layers in NN)
- **Forward & Backward Propagation**
- Forward: Propagate result from previous layer to next layer until output layer reached



Introduction to PyTorch

```
Neural Networks Module [torch.nn]
• Containers
  • Module(torch.nn.Module)
  • Sequential(torch.nn.Sequential)
• Layers
  • Linear Layer (Linear)
  • Linear Layer with no activation (torch.nn.Linear)
  • Non-linear activation functions
  • ReLU(torch.nn.ReLU)
  • Sigmoid(torch.nn.Sigmoid)
  • Softmax(torch.nn.Softmax)

loss_function = torch.nn.MSELoss()
loss_function = torch.nn.BCELoss()
loss_function = torch.nn.CrossEntropyLoss()

optimizer = torch.optim.SGD([w_1, w_2], lr=0.01)
optimizer = torch.optim.Adam([w_1, w_2], lr=0.01)

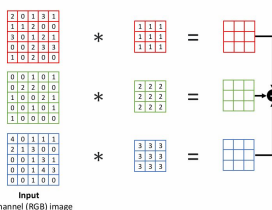
optimizer.zero_grad()
# Set gradients to zero before computing gradient
optimizer.step()
# Update the weights, and let the optimizer know that one step of optimization is done
```

Convolution Neural Networks (CNN)

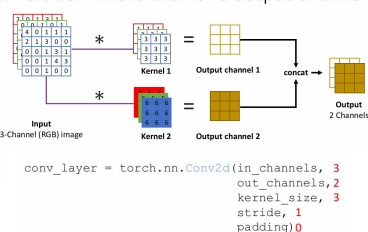
- Convolution: multiply and sum (use padding to get different output size)
- $Y = \frac{X - K + 2P}{S} + 1$, where Y =output, X =input, K =kernel/filter, P =padding, S =stride
- Convolution: More than one input channel
- $R_{kernels} = R_{output\ channels} \times R_{channels\ in\ kernel} \times R_{input\ channels}$

Stride - num of squares moved every iteration of sliding window

Convolution: More than one input channel



Convolution: More than one output channel



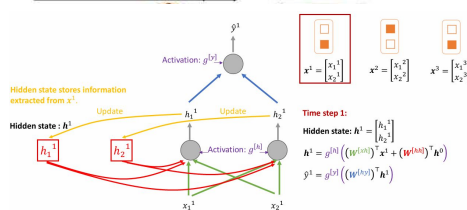
```
conv_layer = torch.nn.Conv2d(in_channels=3, out_channels=2, kernel_size=3, stride=1, padding=0)
```

Receptive field - region of input img that a particular neuron is influenced by

- Feed in batch is computationally efficient, more stable in GD convergence
- Receptive field: $r_j = r_{j-1} + (K_j - 1) \times j_{i-1} \times S_j$ ($r_0 = j_0 = 1$)
- An increased receptive field enables deeper layers to capture global features, spatial context, and high-level structures, improving performance of tasks requiring contextual info
- Pooling layer: downsample feature mapping (max-pool/average-pool/sum-pool)

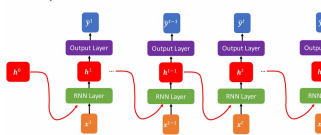
L10: Neural Networks on Sequential Data

- Sequential data: order of the data matters (need contextual information)
- **Recurrent Neural Networks (RNN)**



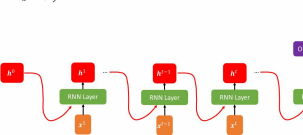
Recurrent Neural Network: Many-to-Many

$$T_x = T_y > 1$$



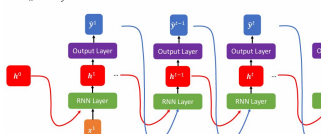
Recurrent Neural Network: Many-to-One

$$T_x > 1, T_y = 1$$



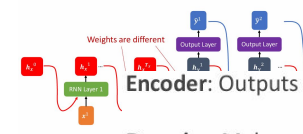
Recurrent Neural Network: One-to-Many

$$T_x = 1, T_y > 1$$

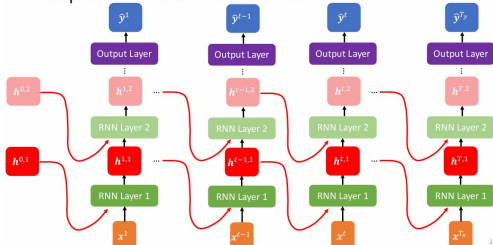


Recurrent Neural Network: Many-to-Many

$$T_x \neq T_y, T_x > 1, T_y > 1$$



Deep Recurrent Neural Networks

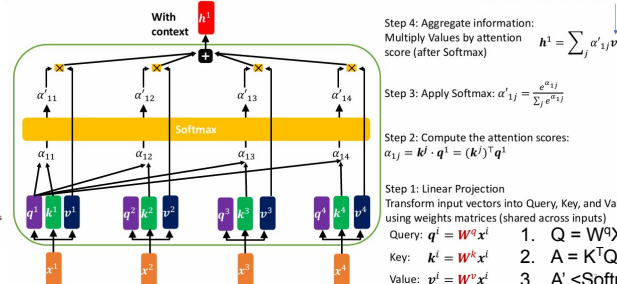


Pro: capture contextual info

Con: prediction at time step t must wait until all prev steps hv been completed

Self-Attention

- Use attention score to check if two inputs are relevant $\alpha = q \cdot k$
- Output of self-attention module is a weighted sum of the value vectors based on the attention scores



Positional encoding explicitly position information into the original input features (e.g., one-hot vectors):

$$PE(i, 2k) = \sin\left(\frac{i}{10000^{2k/d}}\right), \quad PE(i, 2k+1) = \cos\left(\frac{i}{10000^{2k/d}}\right)$$

$$x^{i,l} = x^l + PE^i$$

Unique for each position!

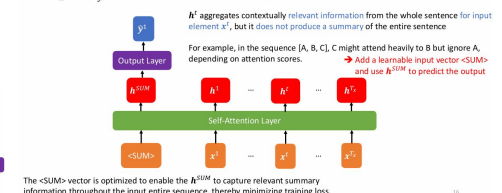
x^l : Original input feature vector for position i , where $x^l \in \mathbb{R}^d$
 PE^i : Positional encoding vector for position i , where $PE^i \in \mathbb{R}^d$
 $x^{i,l}$: Final input feature vector after adding positional encoding.

Issues with Deep Learning

- Overfitting: Regularization
 - Dropout (during training, randomly set some activations to 0 at different timestamps)
 - Early stopping (stop when validation loss starts increasing)
- Vanishing/Exploding Gradient (during backpropagation)
 - Vanishing gradient: small gradients got multiplied repeatedly until almost zero
 - Exploding gradient: large gradients got multiplied repeatedly until it overflows
 - Solution: non-saturating activation functions e.g. ReLU / gradient clipping (clip gradient within a range)

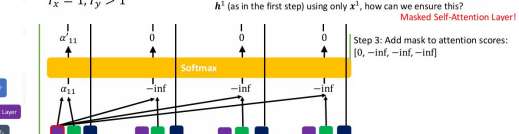
Attention Neural Network: Many-to-One

$$T_x > 1, T_y = 1$$



Attention Neural Network: One-to-Many

$$T_x = 1, T_y > 1$$

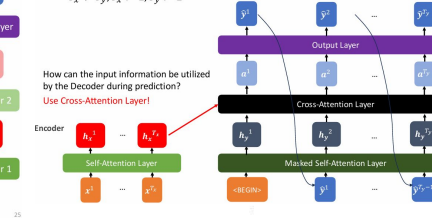


Encoder: Outputs a representation (e.g., vectors) for the input sequence.

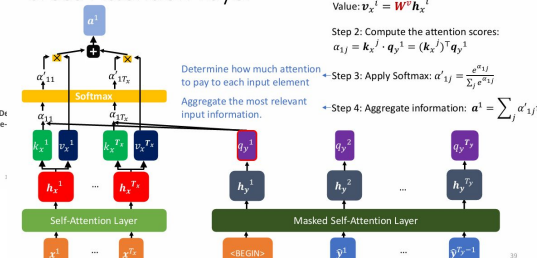
Decoder: Make use of the encoder's output to generate the output sequence.

Attention Neural Network: Many-to-Many

$$T_x \neq T_y, T_x > 1, T_y > 1$$



Cross-Attention Layer



Transformer

- Deep attention neural network
- Basic building blocks: Encoder Block and Decoder Block

